DICTIONARIES

INFO1 Mission 7 (S9)

Jean-Luc Bibaud



Dictionaries

```
{KEY: VALUE, KEY: VALUE, ...}
```

A collection of key : value pairs

1. You look things up **by key**, not by position
2. Each key is **unique**: adding it again replaces the value
3. Keys must be **immutable** (str, int, tuple), so no lists

```
eng2sp = {}                    # create
eng2sp["one"] = "uno"          # add
eng2sp["two"] = "dos"

eng2sp["two"]      → 'dos'
eng2sp["four"]     → KeyError: 'four'
"one" in eng2sp    → True (keys only)
len(eng2sp)        → 2
```

Why dictionaries?

Updating and searching are FAST!

A list: check one by one



Finding "tres" took 3 checks. More items, more checks.

A dictionary: jump straight there



One step, whether the dict has 10 or 10 million items.

Why? Hashing! (you don't need to know the details) The key is turned into a number that says where the value is stored. That's why keys can't change.

Looping

```
eng2sp = {"one": "uno", "two": "dos", "three": "tres"}
```

Keys and values together

```
for (k, v) in eng2sp.items():
        print("Got", k, "that maps to", v)
```

```
→ Got one that maps to uno
  Got two that maps to dos
  Got three that maps to tres
```

The brackets around **`k, v`** are optional: it's tuple unpacking. Order = the order you added them.

Only the values / only the keys

```
for v in eng2sp.values():
        print("Got value", v)
```

```
→ Got value uno
  Got value dos
  Got value tres
```

```
for k in eng2sp.keys():
        print("Got key", k)
```

```
→ Got key one
  Got key two
  Got key three
```

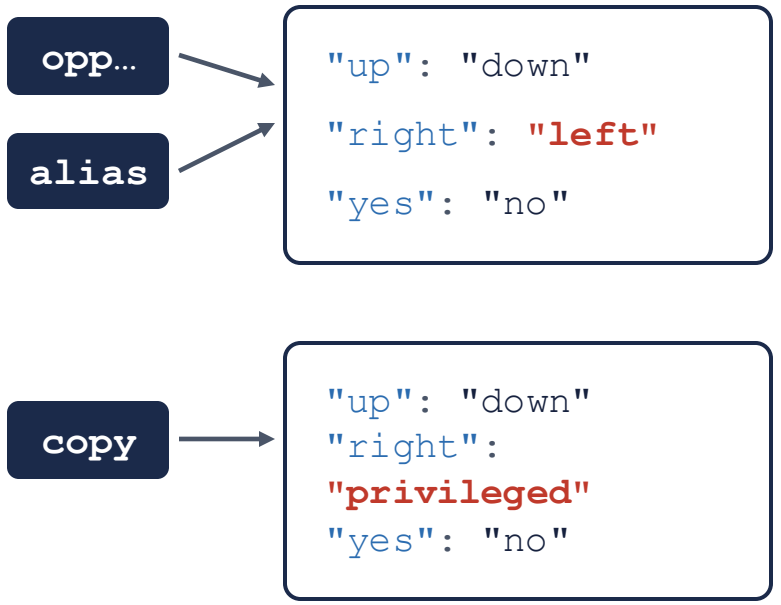
Aliasing and copying

```
opposites = {"up": "down", "right": "wrong",
             "yes": "no"}
alias = opposites              # same dict
copy = opposites.copy()        # a new dict

alias["right"] = "left"
opposites["right"]
→ 'left'    ← changed through alias!

copy["right"] = "privileged"
opposites["right"]
→ 'left'    ← the copy is separate
```

Two names, one dict



Exactly like lists: `=` gives a second name, `.copy()` gives a real copy.

Get for a default

```
DICT.get(KEY, DEFAULT)
```

No more KeyError

```
eng2sp.get("one", "no translation")
→ 'uno'

eng2sp.get("four", "no translation")
→ 'no translation'

eng2sp["four"]       # without get
→ KeyError: 'four'
```

Perfect for counting

```
letter_counts = {}
for c in "Mississippi":
        letter_counts[c] = \
            letter_counts.get(c, 0) + 1

print(letter_counts)
→ {'M': 1, 'i': 4, 's': 4, 'p': 2}
```

Transformation to a list

```
letter_items = list(letter_counts.items())
print(letter_items)
→ [('M', 1), ('i', 4), ('s', 4), ('p', 2)]

letter_items.sort()
print(letter_items)
→ [('M', 1), ('i', 4), ('p', 2), ('s', 4)]

sorted(letter_counts)
→ ['M', 'i', 'p', 's']
```

Good to know

1. **`items()`** gives (key, value) tuples; **`list()`** turns them into a list you can sort.
2. Tuples sort by the first element: the key.
3. Why is **`'M'`** first? Capitals come before lowercase letters.
4. **`.sort()`** changes the list and returns None. **`sorted()`** gives a new list.